FINAL PROJECT REPORT

Vehicle Rental Service

Team: Amir, Tangharyk, Madiyar | 8 November 2026

1. Project overview

Vehicle Rental Service is a Java console application for managing cars, bikes and vans. It supports vehicle registration, rental creation, flexible pricing, optional extras, returns, rate changes and event notifications. Six design patterns separate these responsibilities into small collaborating components.

Objectives

• Create different vehicle types through a common product contract.

• Build validated rentals with a selected pricing policy, promotion and optional extras.

• Track availability and active rentals, then notify customers and analytics listeners when events occur.

Pattern	Main responsibility	Owner
Factory Method	Create Car, Bike and Van products.	Amir
Builder	Validate rental details and construct BaseRental.	Amir
Strategy	Select the tariff calculation algorithm.	Tangharyk
Decorator	Combine insurance, GPS and child-seat fees.	Tangharyk
Observer	Distribute fleet events to subscribed listeners.	Madiyar
Facade	Coordinate the complete rental workflow.	Madiyar

Technology and architecture

The Maven configuration targets Java 21 and uses JUnit Jupiter 5.10.2. Main calls RentalFacade, which coordinates the factory, builder, pricing, decorator and observer packages. The contract package defines the shared interfaces and enums. Data stays in memory; customer SMS messages are simulated in the console.

2. Features and use cases

The implemented requirements follow the operations exposed by RentalFacade and the console demonstration. They describe the current in-memory service rather than a deployed booking website.

Use case	Required input or condition	Successful outcome
Register a vehicle	Vehicle type, unique ID, model and positive finite hourly rate.	A concrete vehicle is stored in the fleet and VEHICLE_REGISTERED is published.
Create a rental	Unique active rental ID, customer, available vehicle, positive hours and pricing strategy.	A Rental is created, stored and returned; its vehicle becomes unavailable.
Select extras	Optional list containing insurance, GPS or child-seat selections.	Each selected extra wraps the rental and adds its fee and description.
Return a vehicle	An existing active rental ID, with surrounding spaces normalized.	The active rental is removed and its vehicle becomes available again.
Change an hourly rate	An existing vehicle ID and a positive finite replacement rate.	The rate changes and a PRICE_DROPPED event is published.
Subscribe to events	A non-null RentalListener.	The listener receives subsequent events until it is unsubscribed.

Typical customer scenario

A customer selects a registered car, a rental duration and a pricing strategy. The facade checks availability, builds the rental and applies chosen extras. When the rental is returned, subscribed customer listeners receive an availability alert and the analytics listener records the event.

Scope boundaries

The program has no login, online booking interface, database, payment gateway or external SMS connection. It demonstrates the rental domain and design-pattern collaboration through Java objects, automated tests and console output.

The rate-change method is named applyPriceDrop, but it does not check that the new rate is lower than the previous rate. Its implemented validation requires a positive finite rate.

3. Architecture and class connections

The project separates shared contracts from specialized implementations. Main demonstrates scenarios; RentalFacade coordinates domain operations; the remaining packages provide focused services. This keeps the client from repeating construction, pricing and notification logic.

Package	Key classes or interfaces	Responsibility
contract	Vehicle, Rental, PricingStrategy, RentalListener and three enums.	Define the types used across package boundaries.
factory	VehicleFactory, concrete creators and vehicle products.	Create cars, bikes and vans with common identity and rate validation.
builder	RentalBuilder and BaseRental.	Collect validated reservation values and create the base rental.
strategy	HourlyPricing, DailyPricing, WeeklyNightPricing.	Calculate the selected time-based tariff.
decorator	RentalDecorator and its three concrete wrappers.	Add independent extra fees while preserving Rental.
observer	AvailabilityPublisher, CustomerNotifier, FleetAnalyticsListener.	Manage subscriptions, deliver events and process callbacks.
facade	RentalFacade.	Own fleet and active-rental maps and coordinate workflows.

Important retained references

BaseRental retains its Vehicle and PricingStrategy. A RentalDecorator retains another Rental, so several extras form a wrapper chain. AvailabilityPublisher retains RentalListener objects. RentalFacade retains registered vehicles, active rentals and its publisher.

Why the shared contracts matter

A strategy can calculate a price for any Vehicle implementation. A decorator can wrap BaseRental or another decorator through Rental. The publisher can notify any RentalListener implementation without knowing its concrete behavior. The facade still knows concrete factories and decorators where it composes the system.

4. Factory Method

Problem: Fleet registration must create several vehicle types without repeating concrete constructor logic throughout the client.

Solution: VehicleFactory declares createVehicle(...). CarFactory, BikeFactory and VanFactory override it. The common orderNewVehicle(...) operation calls that factory method and returns an available Vehicle.

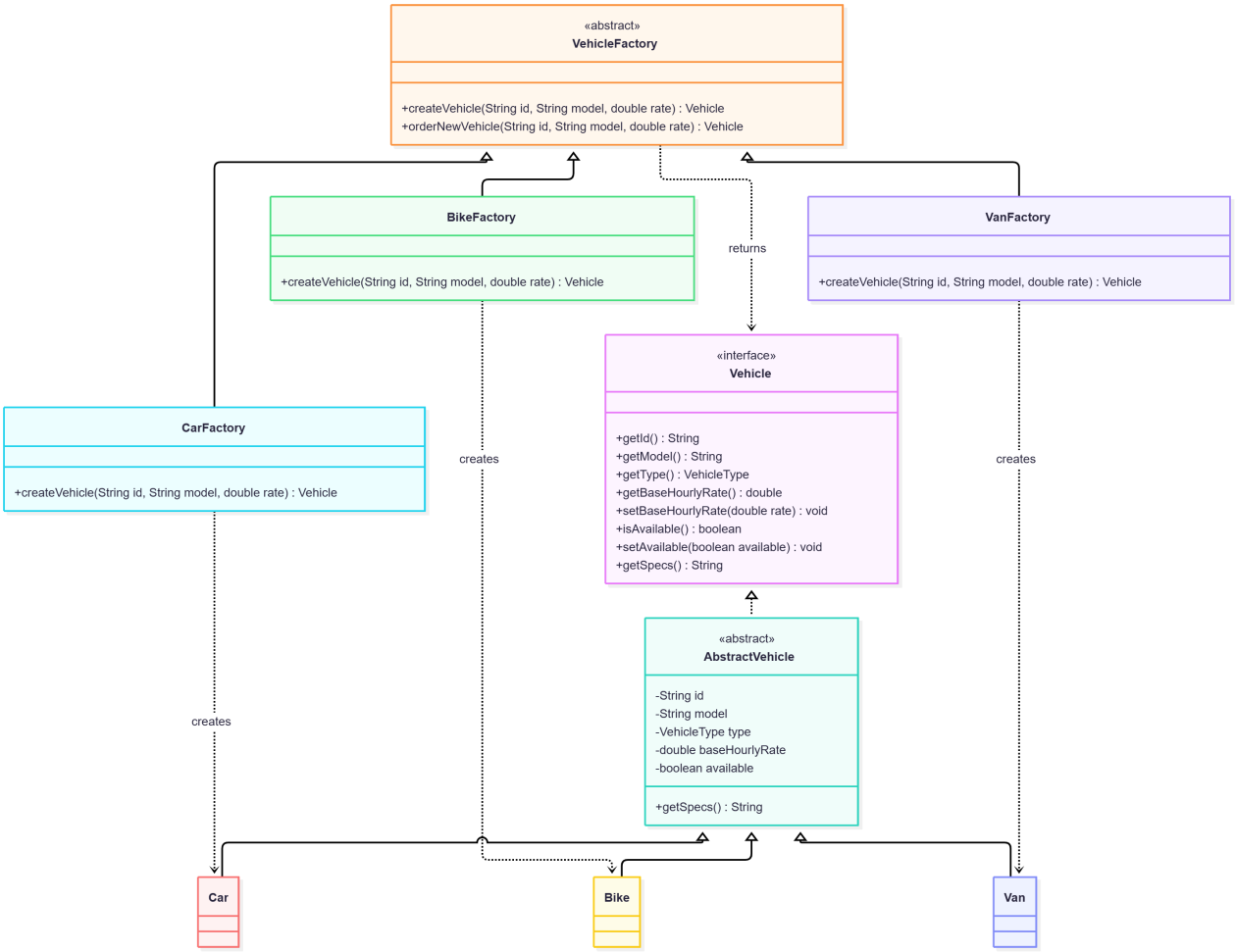


Figure 1. Factory Method: creator and product hierarchies.

Role	Implementation
Product contract	Vehicle
Shared product state	AbstractVehicle
Concrete products	Car, Bike and Van
Creators	VehicleFactory and its three subclasses

AbstractVehicle validates identity, model and a finite positive hourly rate. RentalFacade selects the creator using VehicleType and checks duplicate IDs. New products can reuse the creator hierarchy, although facade selection must also be extended.

5. Builder

Problem: Rental creation requires an ID, customer, vehicle, duration and pricing strategy, plus optional promotion and notes. A long positional constructor would be difficult to read and validate.

Solution: RentalBuilder provides seven fluent with... methods. build() checks the required values, trims text and constructs BaseRental through the Rental interface.

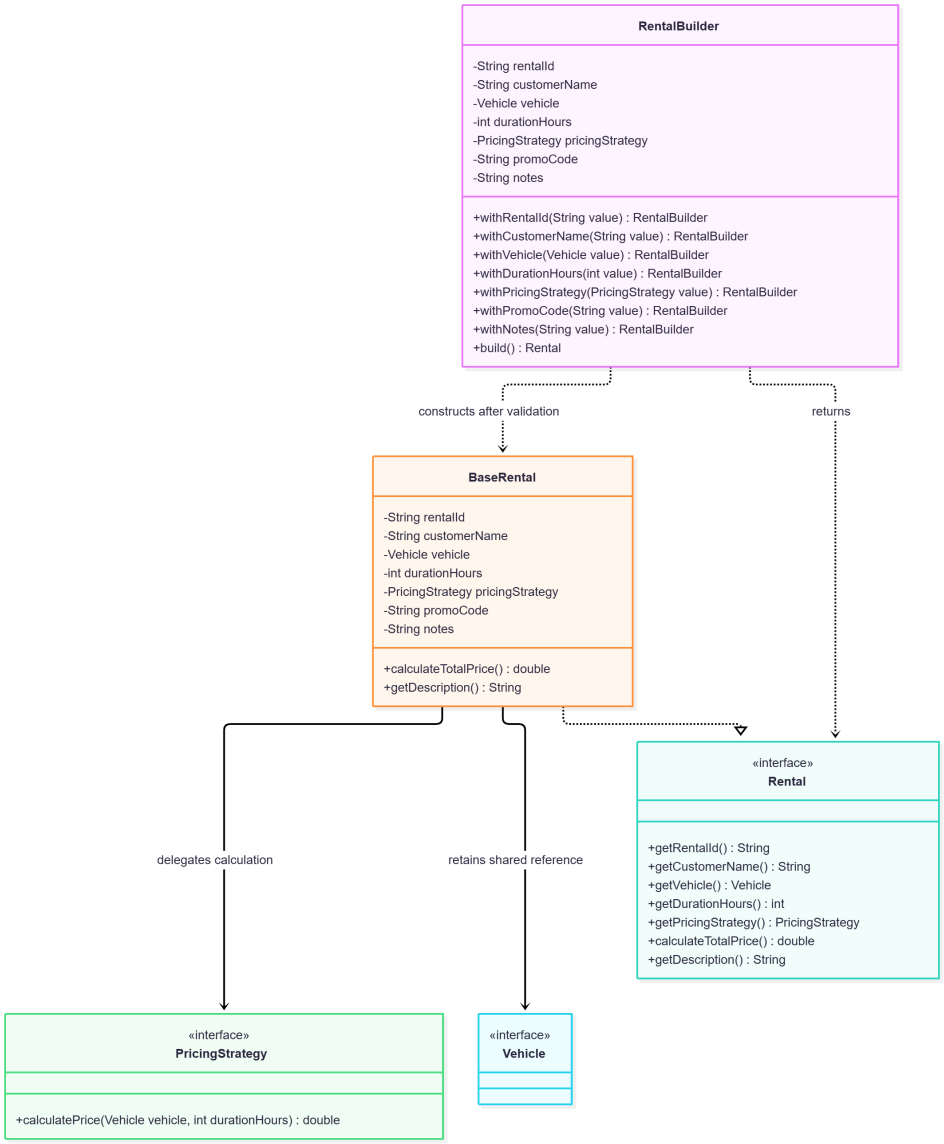


Figure 2. RentalBuilder constructs BaseRental and supplies shared dependencies.

Missing required values or a non-positive duration cause IllegalStateException. Blank optional values become null. Product fields are final, but the referenced Vehicle remains mutable; later rate changes can affect future price calculations.

6. Strategy: rental pricing

Problem: Different rental durations need different tariffs. Keeping every tariff inside the rental class would make pricing changes harder to isolate.

Solution: BaseRental delegates calculation to PricingStrategy. HourlyPricing, DailyPricing and WeeklyNightPricing implement the same contract and can be selected when the rental is built.

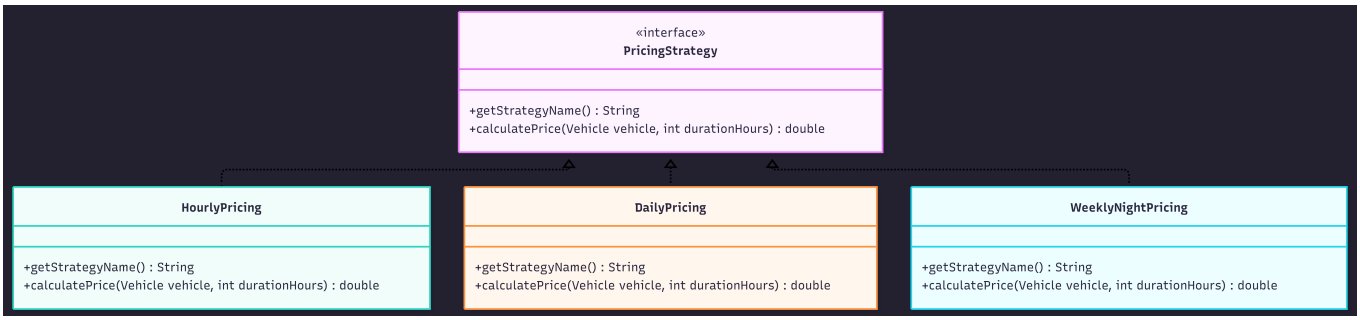


Figure 3. PricingStrategy and its interchangeable implementations.

Strategy	Implemented rule
HourlyPricing	Hourly rate × duration in hours.
DailyPricing	15% discount on the hourly total when duration is at least 24 hours; otherwise the hourly total.
WeeklyNightPricing	30% discount at 168 hours or more; otherwise 10% at 12 hours or more; otherwise the hourly total.

Promotion and calculation order

The strategies round their results to two decimal places. BaseRental then applies a 10% discount when the optional code AITU10 is supplied, ignoring letter case. Extra fees are added afterwards by decorators.

Worked example

For a car at $8 per hour rented for 48 hours, DailyPricing gives $384.00 × 0.85 = $326.40. The AITU10 promotion reduces the base rental to $293.76 before extras.

WeeklyNightPricing uses duration thresholds; it does not inspect actual nighttime hours or calendar dates.

7. Decorator: optional extras

Problem: Customers can select different combinations of extras. A separate rental subclass for every combination would duplicate behavior.

Solution: RentalDecorator implements Rental and retains a wrapped Rental. Each concrete decorator delegates the existing behavior, adds its fee and extends the description. Wrappers can be stacked.

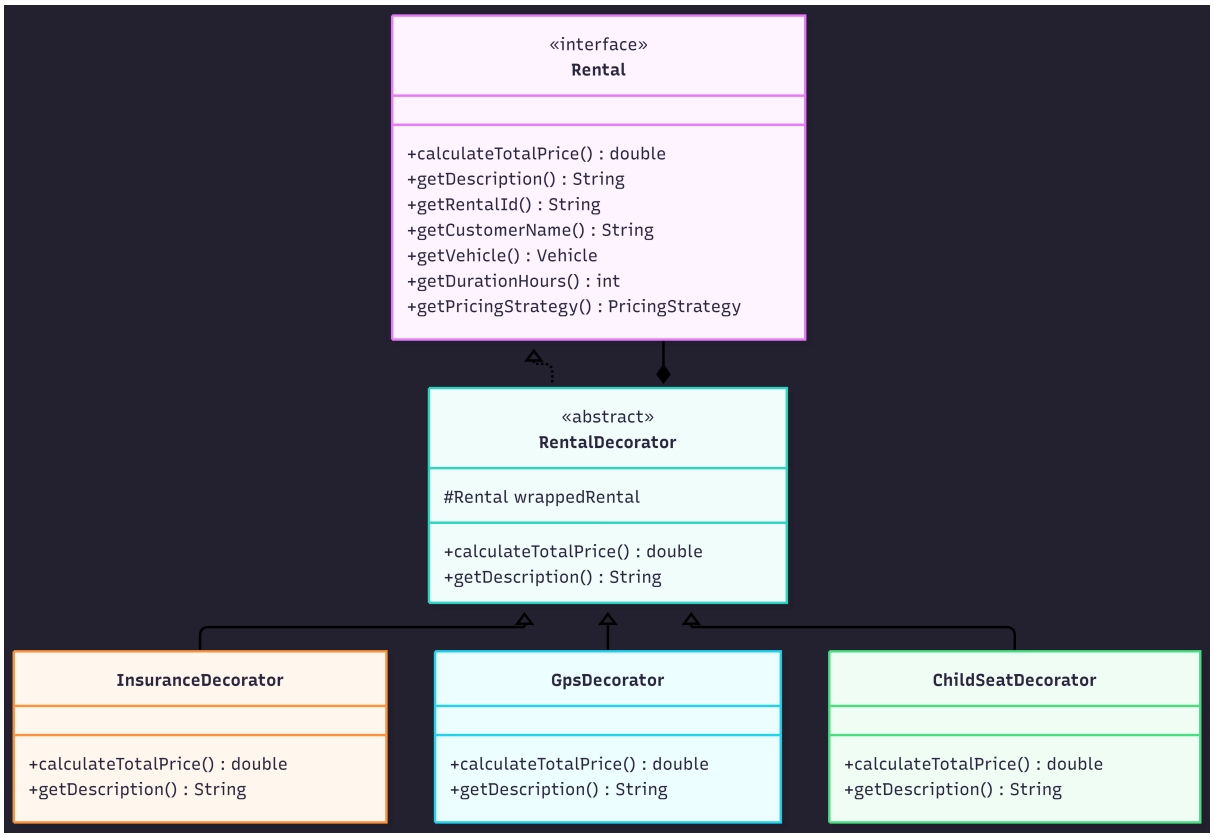


Figure 4. Optional extras wrap the common Rental interface.

Extra	Fee
InsuranceDecorator	$15 per started 24-hour period, with at least one period.
GpsDecorator	$10 flat fee.
ChildSeatDecorator	$12 flat fee.

In the 48-hour car example, insurance adds $30 and a child seat adds $12, producing $335.76. The facade applies extras in list order; repeated extras are not automatically removed.

The supplied UML uses a composition symbol for the wrapped rental. Java retains a reference here and does not enforce exclusive ownership.

8. Observer and Facade

Observer: AvailabilityPublisher manages RentalListener subscriptions and synchronously notifies a snapshot of that list. CustomerNotifier records and prints availability/price-drop messages; FleetAnalyticsListener counts every supported event.

Facade: RentalFacade exposes registration, rent, return and rate-change operations. It owns fleet and active-rental maps, selects factories, configures the builder, applies decorators and publishes lifecycle events.

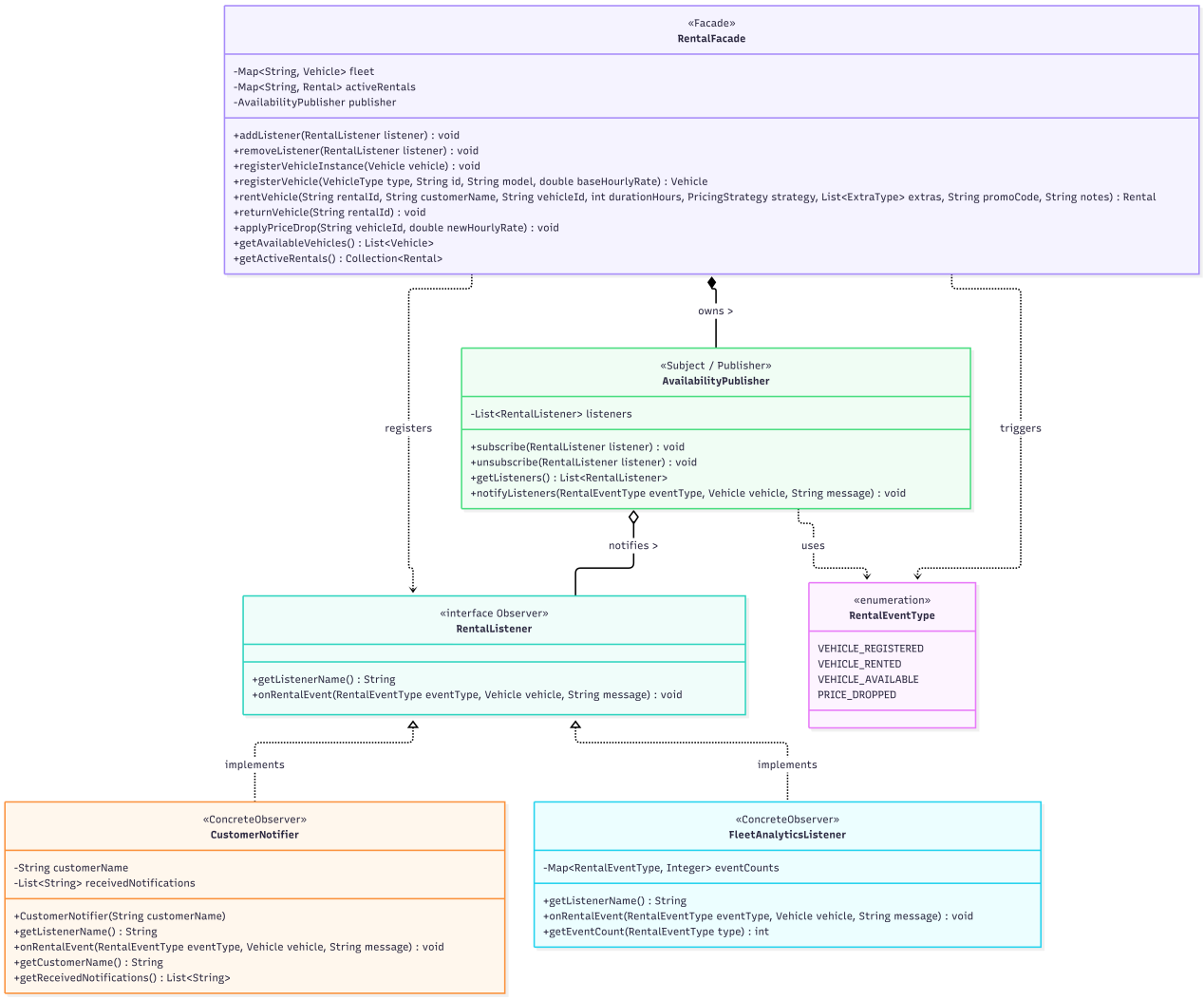


Figure 5. RentalFacade, event publisher and concrete listeners.

Events are VEHICLE_REGISTERED, VEHICLE_RENTED, VEHICLE_AVAILABLE and PRICE_DROPPED. Returning a rental restores availability and removes the active rental. Customer notifications are console simulations rather than messages sent through an external SMS provider.

9. Rental creation sequence

A sequence diagram shows the order of calls during one operation. This successful flow starts after the vehicle has already been registered through a factory.

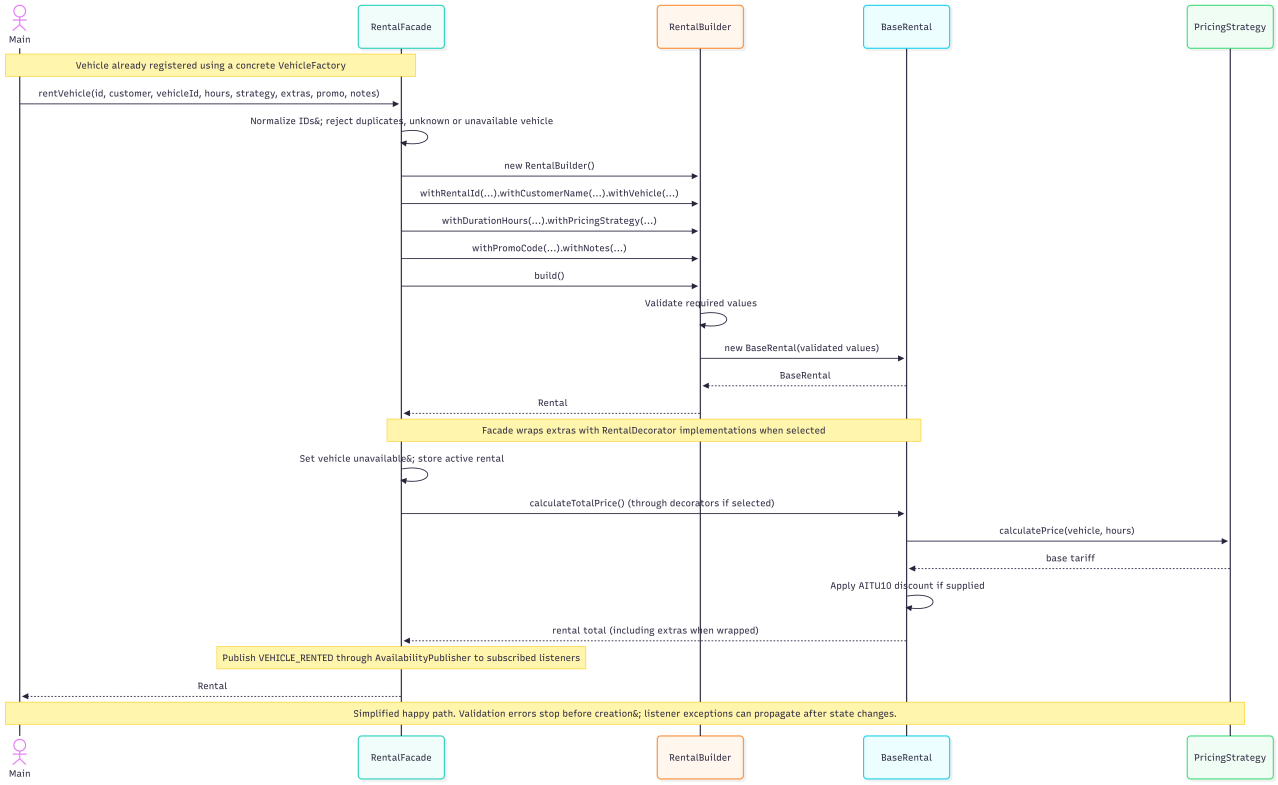


Figure 6. Creating a rental through RentalFacade.

• Main submits the rental details. The facade normalizes IDs and checks uniqueness, vehicle existence and availability.

• RentalBuilder validates the values and creates BaseRental with the chosen Vehicle and PricingStrategy.

• The facade wraps any selected extras, marks the vehicle unavailable and stores the active rental.

• Price calculation passes through the wrappers to BaseRental and its strategy. The promotion applies before extra fees.

• The facade publishes VEHICLE_RENTED and returns the Rental to Main.

10. Return and notification sequence

returnVehicle(...) removes the active rental, marks its vehicle available and publishes VEHICLE_AVAILABLE. Each subscribed listener receives one callback. Customer listeners print alerts, while the analytics listener increments its event count.

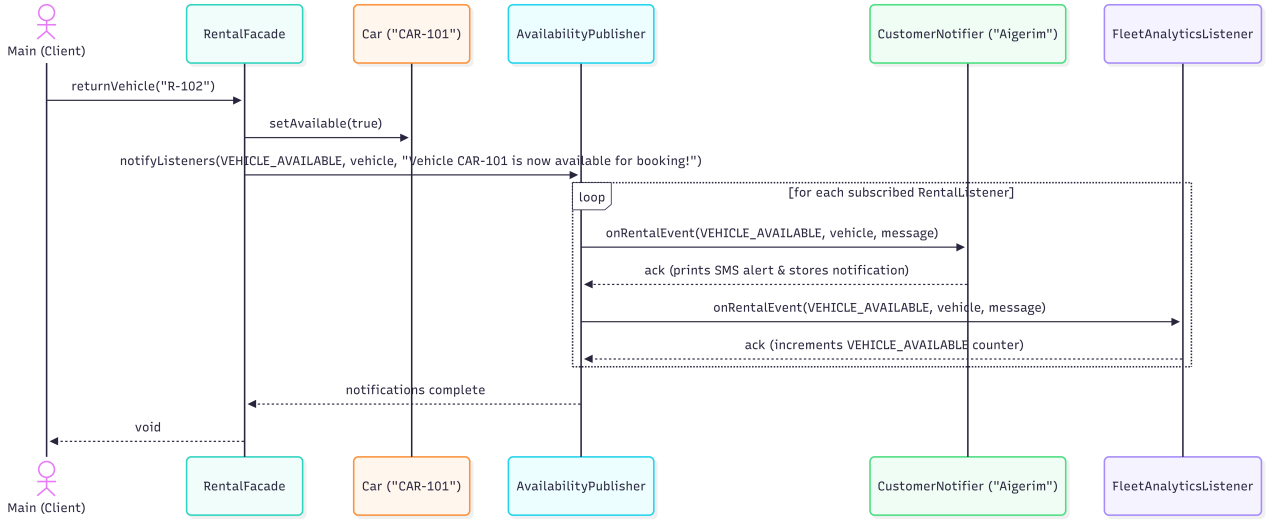


Figure 7. Observer notification after a vehicle is returned.

```
--- [SCENARIO 5: OBSERVER REAL-TIME NOTIFICATIONS (VEHICLE RETURN & PRICE DROP)] ---
>> Returning rental R-102 (Toyota Camry 70)...
[SMS to Aigerim] Event VEHICLE_AVAILABLE for Toyota Camry 70: Vehicle CAR-101 is now available for booking!
[SMS to Nurlan] Event VEHICLE_AVAILABLE for Toyota Camry 70: Vehicle CAR-101 is now available for booking!
[Analytics] Recorded VEHICLE_AVAILABLE (Total: 1)

>> Applying special weekend price drop on VAN-303 ($14.00/h -> $11.50/h)...
[SMS to Aigerim] Event PRICE_DROPPED for Mercedes Sprinter: Hourly rate dropped to $11.50/h!
[SMS to Nurlan] Event PRICE_DROPPED for Mercedes Sprinter: Hourly rate dropped to $11.50/h!
[Analytics] Recorded PRICE_DROPPED (Total: 1)

--- [FLEET ANALYTICS SUMMARY] ---
VEHICLE_REGISTERED : 3
VEHICLE_RENTED     : 3
VEHICLE_AVAILABLE  : 1
PRICE_DROPPED      : 1
================================================================================

Process finished with exit code 0
```

Figure 8. Supplied console evidence: return alerts, price-drop alerts and analytics.

The demo finishes with 3 registrations, 3 rentals, 1 availability event and 1 price-drop event. The supplied sequence simplifies the callback loop; its return arrows represent control returning from void callbacks.

11. Pricing examples and boundaries

The following comparisons use a vehicle with an hourly rate of $10, before promotions or extras. They demonstrate the actual duration thresholds implemented by the three strategies.

Hours	HourlyPricing	DailyPricing	WeeklyNightPricing
1	$10.00	$10.00	$10.00
12	$120.00	$120.00	$108.00
23	$230.00	$230.00	$207.00
24	$240.00	$204.00	$216.00
25	$250.00	$212.50	$225.00
168	$1,680.00	$1,428.00	$1,176.00

At a discount threshold, a longer rental can cost less than a slightly shorter one: DailyPricing charges $230 for 23 hours but $204 for 24 hours at this rate. This follows the current percentage-discount policy; it is not daily-block rounding.

Complete 48-hour car calculation

Stage	Amount
48 hours at $8 per hour	$384.00
Apply the 15% DailyPricing discount	$326.40
Apply AITU10 to the discounted base rental	$293.76
Add insurance: two started days at $15	$323.76
Add the $12 child seat	$335.76

Insurance periods and later rate changes

Insurance costs $15 at both 1 hour and 24 hours, $30 at 25 hours, and $105 at 168 hours. GPS and child-seat charges stay flat regardless of duration.

BaseRental does not store a fixed price snapshot. If the same referenced vehicle later receives a different hourly rate, another calculateTotalPrice() call uses that updated rate. A production system should define whether active bookings keep their original price.

12. Validation and rental state

Validation is split between product creation, rental construction and facade workflows. Invalid requests are rejected at the layer that owns the relevant rule. The current tests distinguish invalid arguments from an unavailable vehicle or incomplete builder state.

Condition	Checking layer	Expected exception
Blank vehicle ID/model, null type, or non-positive/non-finite rate.	AbstractVehicle / vehicle creation.	IllegalArgumentException
Duplicate fleet ID, including a whitespace-normalized duplicate.	RentalFacade registration.	IllegalArgumentException
Blank rental ID, duplicate active rental ID, or unknown vehicle.	RentalFacade rental request.	IllegalArgumentException
Vehicle is already unavailable.	RentalFacade rental request.	IllegalStateException
Missing builder values or non-positive duration.	RentalBuilder.build().	IllegalStateException
Unknown active rental on return.	RentalFacade return operation.	IllegalArgumentException

Successful lifecycle transitions

Operation	Vehicle availability	Active-rental collection
Register a new vehicle	Available.	No rental added.
Rent an available vehicle	Unavailable.	New rental stored.
Return an active rental	Available again.	Returned rental removed.

IDs and required customer text are trimmed. Blank optional promo codes and notes become null. Null extra entries are ignored; repeated non-null extras are applied repeatedly. These behaviors are part of the current implementation.

State changes occur before observer notification. Listener exceptions are not isolated and can propagate after state has changed, so the workflow is not transactional. Concurrent rentals are also not protected by locking.

13. Integration and SOLID principles

All six patterns collaborate through shared contracts. Factory Method produces Vehicle objects; Builder produces a Rental; Strategy computes its base price; Decorator adds extras; Observer distributes events; and Facade coordinates the workflow.

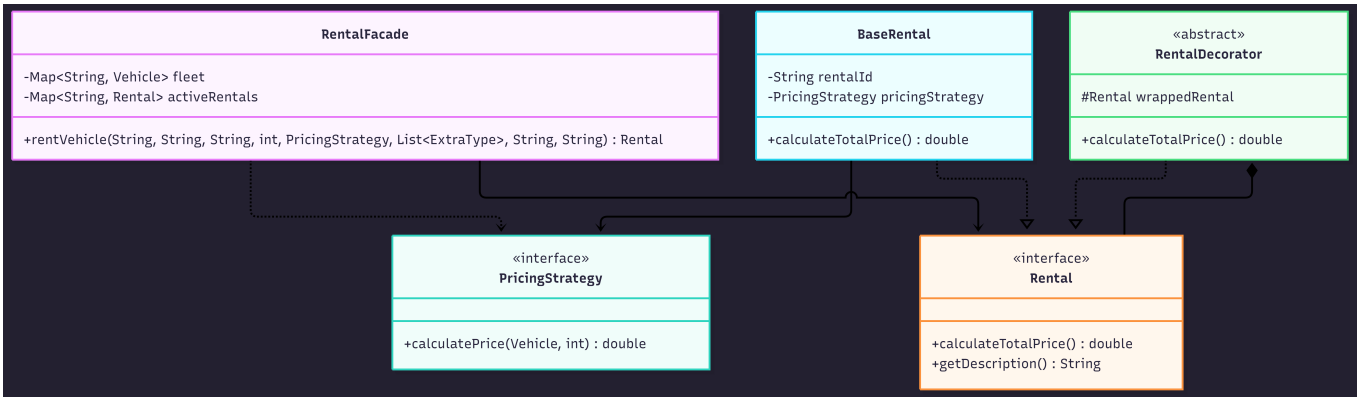


Figure 9. Supplied subsystem integration view: Rental, pricing and decorators.

Principle	Concrete application
SRP	Factories create products, the builder validates construction, strategies calculate tariffs and the publisher dispatches events.
OCP	New strategies, decorators and listeners can reuse existing abstractions. Facade enum switches still need updates for newly selected vehicle types or extras.
LSP	Vehicle products share the Vehicle contract; BaseRental and its wrappers share Rental and preserve the common API.
ISP	Vehicle, Rental, PricingStrategy and RentalListener separate fleet, reservation, pricing and notification capabilities.
DIP	BaseRental uses Vehicle and PricingStrategy; decorators wrap Rental; the publisher uses RentalListener. The facade still constructs concrete factories and decorators.

The supplied integrated UML focuses on the pricing/decorator subsystem. The class diagrams on earlier pages show the factory, builder and observer boundaries. The design localizes changes but does not eliminate every dependency on concrete classes.

14. Testing and verification strategy

The project includes nine JUnit test classes containing 43 test methods. Unit tests isolate each pattern; facade and integration tests check the resulting state and interactions. The table counts methods in the supplied source, not an estimated coverage percentage.

Test class	Tests	Main focus
VehicleFactoryTest	8	Product types/specifications, rate validation, independent products and the overridden factory method.
RentalBuilderTest	8	Required values, optional text, promotion, builder reuse and pricing delegation.
PricingStrategyTest	4	Hourly multiplication, discount thresholds and invalid input.
RentalDecoratorTest	4	Insurance periods, flat extras, stacked wrappers and null wrapped rentals.
ObserverPatternTest	5	Subscription, unsubscription, notification history, event counts and duplicate subscriptions.
RentalFacadeUnitTest	9	Registration, rental/return, rate changes, canonical IDs and duplicate rejection.
FactoryBuilderIntegrationTest	3	Vehicle creation and validated rentals through the facade, plus invalid request state preservation.
FullRentalFlowIntegrationTest	1	End-to-end rental flow combining strategies and decorators.
MainTest	1	The console demonstration runs without an exception.

How to assess the evidence

The supplied Maven screenshot on the next page reports all 43 tests passing with zero failures and errors. The previous exception-type mismatches are corrected in the latest test source. During report review, production compilation and Main execution also succeeded in the available Java 17 compatibility check. The configured Java 21 Maven suite was not independently rerun in that environment.

Useful checks before release

• Run the full suite from a clean checkout using the configured JDK 21 and confirm that all 43 tests are discovered.

• Run Main and compare the three prices and four event totals with the reported examples.

• If implementation rules change, update the tests and report together so the submitted evidence describes the same version.

15. Results and conclusion

The source contains 43 test methods across nine test classes. They cover factories, builder validation, tariffs, extras, notifications, facade state changes, integrated rental flows and the main demo. The supplied Maven screenshot reports 43 tests run, 0 failures, 0 errors, 0 skipped and BUILD SUCCESS.

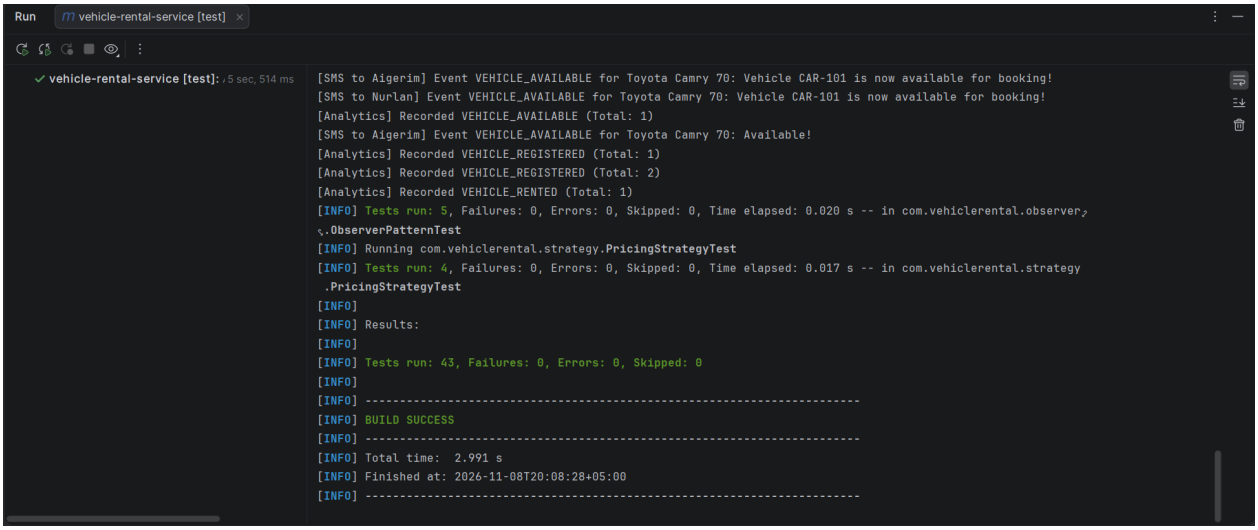


Figure 10. Team-supplied Maven/JUnit result for the final project.

Demo scenario	Result
Bike: 5 hours at $3/hour + GPS	$25.00
Car: 48 hours at $8/hour, DailyPricing, AITU10, insurance + child seat	$335.76
Van: 168 hours at $14/hour, WeeklyNightPricing, insurance + GPS	$1,761.40



Figure 11. Supplied console output confirms the three rental examples.

The latest tests now match the facade exception types for unknown vehicles and blank rental IDs. Production sources compiled and Main ran during this report review using the available Java 17 compatibility check; the Java 21 Maven suite was not independently rerun here.

Conclusion and future work

The project demonstrates six cooperating patterns with validated rental creation, configurable pricing, composable extras and event-driven notifications. Future work can add persistent storage, fixed price snapshots with decimal money, maintenance states, concurrency control and isolated listener failures. The current version stores data in memory, retains mutable vehicle references and does not provide real payments or external SMS delivery.